Universidad Católica de Oriente · Facultad de Ingeniería
Compiladores — Quiz 4 / Entrega 4 · Semestre 2-2026

Paisascript

Gramática BNF ampliada con reglas semánticas
e implementación del análisis semántico

Lenguaje fuente: Paisascript
Lenguaje destino: Gleam
Integrantes: Andres Felipe Velez · Angie Amaya Fernandez · Julian David Guarnizo Castrillon
Repositorio: github.com/JulianG08/Analizador-lexico
App en vivo: compiladorlex.streamlit.app
Bonus: Modalidad A — IA como capa adicional de validación semántica

1. Gramática ampliada con reglas semánticas

Este documento extiende la gramática libre de contexto de Paisascript usada en las entregas anteriores. Las **producciones sintácticas originales** se muestran en rojo. Las **reglas semánticas añadidas** se muestran en texto negro sobre fondo ámbar e indican, para cada regla, los atributos involucrados (sintetizados o heredados) y el nodo al que pertenecen, el dominio de valores o restricción que debe satisfacerse, y las acciones que ejecuta el compilador para validarla.

> `<produccion> ::= ...` — regla original de la gramática (producción sintáctica sin ampliar).
> REGLA SEM-x — regla semántica añadida: atributos, restricción y acciones de validación.

Las reglas semánticas **exceden** a las sintácticas: no describen el orden de los elementos en la cadena de entrada, sino propiedades sobre los valores que esos elementos representan. El compilador las valida en una fase independiente (*analizador_semantico.py*) que recorre el árbol de sintaxis y construye una tabla de símbolos.

1.1 Resumen de las ocho reglas

Regla	Nodo(s) de la gramática	Atributos	Restricción que se valida
SEM-1	<sent_declaracion>	tipo_declarado (heredado), type(expr) (sintetizado)	El tipo anotado coincide con el tipo inferido de la expresión.
SEM-2	<expr_primaria>, <sent_lectura>	id.lexema (heredado), id.type e id.declarado (sintetizados)	Todo identificador usado está declarado en el ámbito actual o ancestro.
SEM-3	Producciones de expresiones	type_izq, type_der, type (sintetizados)	Operandos compatibles: mismo tipo aritmético; lógicos y condiciones siono.
SEM-4	<sent_si>, <sent_mientras>, <sent_repita>	type(cond) (sintetizado), cond_esperado (heredado)	La condición de control es de tipo siono.
SEM-5	<sent_llamada>, <sufijo_id>	arity, arg_type[], param_type[], return_type	La función existe, la aridad coincide y cada argumento tiene el tipo del parámetro.
SEM-6	<sent_mientras>, <sent_repita>	leidas(cond), redefinidas(cuerpo), valor(cond) (sintetizados); continua_si (heredado)	El cuerpo redefine alguna variable de la condición: el bucle puede terminar.
SEM-7	<sent_para> y su <bloque>	var_control.escribible (heredado), categoria y type (sintetizados)	La variable de control no se puede reenlazar dentro del bucle.
SEM-8	<sent_para>, <paso_opcional>	valor(desde), valor(hasta), valor(paso), direccion (sintetizados); paso_implicito (heredado)	El paso no es 0 y su signo concuerda con los límites: el recorrido existe.

1.2 Los tres bucles de Paisascript y las reglas que los gobiernan

Gleam **no tiene bucles**: no hay `while`, `for`, `break` ni `continue`; solo `case` y recursión. Paisascript sí los ofrece, porque son requisito del curso y porque su traducción es mecánica. Las tres formas cubren los tres patrones clásicos de repetición y cada una se apoya en una construcción distinta de Gleam:

Bucle en Paisascript	Clase	Traducción a Gleam	Reglas que lo validan
`mientras_que c dele_pues B hasta_ahi`	Condición al principio (*pre-test*); puede ejecutar cero veces.	Función auxiliar **recursiva de cola** cuyos parámetros son las variables que el cuerpo redefine; `case c { True -> bucle(estado'), False -> estado }`.	SEM-4 (condición siono) y **SEM-6** (progreso).

`pa_cada i desde a hasta b [de_a p] dele_pues B listo_pues`	Contador con límites y paso; recorrido conocido de antemano.	Con paso 1: `list.range(a, b) |> list.each(fn(i) { B })`. Con paso ≠ 1: recursión de cola con incremento, porque `list.range` no admite paso.	SEM-3 (límites numerito), **SEM-7** (variable de control de solo lectura) y **SEM-8** (paso y dirección).
`repita_pues B hasta_que c`	Condición al final (*post-test*); ejecuta al menos una vez.	La misma recursión de cola, con el cuerpo emitido **antes** del `case`: `True -> estado` (caso base), `False -> bucle(estado')`.	SEM-4 (condición siono) y **SEM-6** (progreso, con `continua_si = falso`).

Que los tres bucles se traduzcan a recursión es exactamente lo que vuelve necesarias a SEM-6, SEM-7 y SEM-8: en un lenguaje con `while` de verdad un bucle sin progreso solo se cuelga en ejecución, mientras que aquí genera una función recursiva que nunca alcanza su caso base. El análisis semántico detecta en tiempo de compilación lo que el lenguaje destino no podría ni expresar.

2. Producciones BNF con las reglas semánticas intercaladas

Se reproduce la sección de producciones de la gramática. Cada regla semántica aparece inmediatamente después del nodo que restringe.

```
  Ordenadas de <programa> (arriba) a las expresiones atomicas (abajo).
  Sin recursividad por la izquierda y factorizadas por la izquierda.
  Los terminales literales van entre comillas o en minuscula sin <>.
  e denota la cadena vacia (epsilon).
================================================================================
/* ------------------------------------------------------------------ */
/* 3.1  Programa y declaraciones                                      */
/* ------------------------------------------------------------------ */
<programa>              ::= <lista_declaraciones> $

<lista_declaraciones>   ::= <declaracion> <lista_declaraciones>
                          | e

<declaracion>           ::= <def_funcion>
                          | <sentencia>

/* ------------------------------------------------------------------ */
/* 3.2  Definicion de funciones                                       */
/* ------------------------------------------------------------------ */
<def_funcion>           ::= hagale_pues IDENTIFICADOR
                            "(" <parametros> ")"
                            <tipo_retorno>
                            dele_pues
                                <bloque>
                            ya_quedo

<tipo_retorno>          ::= pa_que_lleve <tipo>
                          | e

<parametros>            ::= <param_lista>
                          | e

<param_lista>           ::= <tipo> IDENTIFICADOR <param_resto>

<param_resto>           ::= "," <tipo> IDENTIFICADOR <param_resto>
                          | e

<tipo>                  ::= numerito
                          | quebradito
                          | cuento
                          | siono

/* ------------------------------------------------------------------ */
/* 3.3  Bloques y sentencias                                          */
/* ------------------------------------------------------------------ */
<bloque>                ::= <sentencia> <lista_sentencias>

<lista_sentencias>      ::= <sentencia> <lista_sentencias>
                          | e
```

```
<sentencia>             ::= <sent_declaracion>
                          | <sent_lectura>
                          | <sent_impresion>
                          | <sent_si>
                          | <sent_mientras>
                          | <sent_para>
                          | <sent_repita>
                          | <sent_pillemos>
                          | <sent_retornar>
                          | <sent_llamada>

/* Declaracion / asignacion. El tipo es opcional (se infiere).       */
<sent_declaracion>      ::= pille_pues <tipo_opcional> IDENTIFICADOR
                            "=" <expresion>
```

REGLA SEM-1 — Declaración de variable y compatibilidad de tipos

Nodos de la gramática: <sent_declaracion>, <tipo_opcional>, <expresion>

Atributos involucrados	**tipo_declarado**: *heredado* desde <tipo_opcional>. Dominio: {numerito, quebradito, cuento, siono} ∪ {⊥}, donde ⊥ = sin anotación. **type(expr)**: *sintetizado* en <expresion>. Dominio: {numerito, quebradito, cuento, siono} ∪ {error}. **tipo_simbolo**: *sintetizado* en <sent_declaracion>. Es el tipo con el que el identificador entra a la tabla de símbolos.
Restricción	Si **tipo_declarado ≠ ⊥**, entonces **type(expr) = tipo_declarado**. Si **tipo_declarado = ⊥**, se infiere **tipo_simbolo = type(expr)** (como el *let* de Gleam).
Acciones del compilador	1. Calcular type(expr) recorriendo la expresión (atributo sintetizado). 2. Comparar con tipo_declarado. Si difieren, reportar error con **fila, columna, lexema** y mensaje descriptivo. 3. Insertar el símbolo en el **ámbito actual** con su tipo, estado de inicialización = verdadero y posición de declaración. 4. El shadowing (redeclaración en el mismo ámbito) es válido: crea un enlace nuevo, no modifica el anterior. 5. Continuar el análisis (recuperación).

```
<tipo_opcional>         ::= <tipo>
                          | e

<sent_lectura>          ::= escuche_pues "(" IDENTIFICADOR ")"
```

REGLA SEM-2 — Uso de identificadores: declaración previa y visibilidad

Nodos de la gramática: <expr_primaria> (IDENTIFICADOR), <sent_lectura>

Atributos involucrados	**id.lexema**: *heredado* del token IDENTIFICADOR. **id.type**: *sintetizado* desde la tabla de símbolos. Dominio: {numerito, quebradito, cuento, siono, desconocido} ∪ {error}. **id.declarado**: *sintetizado*. Dominio: {verdadero, falso}.
Restricción	**id.declarado = verdadero**: todo identificador usado en una expresión debe existir en el ámbito actual o en alguno de sus ancestros (alcance léxico anidado). **escuche_pues(id)** declara el identificador si no existe y lo marca como inicializado.
Acciones del compilador	1. Recorrer la pila de ámbitos de dentro hacia afuera buscando el lexema (resolución de nombres). 2. Si no aparece, reportar «identificador no declarado» con fila, columna y lexema. 3. Si aparece, sintetizar id.type con el tipo del símbolo; si es una función usada sin paréntesis, reportarlo. 4. En <sent_lectura>, declarar/inicializar el identificador y continuar.

```
<sent_impresion>        ::= hable_pues "(" <expresion> ")"

<sent_retornar>         ::= entregue_pues <expresion>

/* Llamada a funcion usada como sentencia (se descarta su valor).    */
<sent_llamada>          ::= IDENTIFICADOR "(" <argumentos> ")"
```

```
/* ---------------------------------------------------------------- */
/* 3.4  Condicional  (sin recursividad izquierda)                   */
/* ---------------------------------------------------------------- */

<sent_si>               ::= si_acaso <expresion> entonces_pues
                                <bloque>
                            <rama_sino>
                            asi_quedo

<rama_sino>             ::= sino_pues <bloque>
                          | e

/* ---------------------------------------------------------------- */
/* 3.5  Repeticion por condicion  (while)                           */
/* ---------------------------------------------------------------- */

<sent_mientras>         ::= mientras_que <expresion> dele_pues
                                <bloque>
                            hasta_ahi
```

REGLA SEM-4 — Condición de control de tipo booleano

Nodos de la gramática: <sent_si>, <sent_mientras>, <sent_repita>

Atributos involucrados	**type(cond)**: *sintetizado* en la expresión de control. Dominio: tipos del lenguaje ∪ {desconocido, error}. **cond_esperado**: *heredado* desde la construcción. Dominio: {siono}.
Restricción	**type(cond) = siono** en **si_acaso**, en **mientras_que** y en el **hasta_que** de **repita_pues**. Una condición numerito, cuento o quebradito no es válida. En **repita_pues** la condición se evalúa en el ámbito del cuerpo, así que puede leer las variables que el cuerpo creó.
Acciones del compilador	1. Evaluar el tipo sintetizado de la condición. 2. Compararlo con siono. 3. Si no coincide, reportar «la condición debe ser siono» con fila, columna y lexema. 4. Analizar igualmente ambos bloques (recuperación).

```
/* ---------------------------------------------------------------- */
/* 3.6  Repeticion con variable de control  (for)                   */
/* ---------------------------------------------------------------- */

<sent_para>             ::= pa_cada IDENTIFICADOR
                            desde <expresion>
                            hasta <expresion>
                            <paso_opcional>
                            dele_pues
                                <bloque>
                            listo_pues
```

REGLA SEM-7 — La variable de control del pa_cada es de solo lectura

Nodos de la gramática: <sent_para>, y dentro de su <bloque>: <sent_declaracion>, <sent_lectura>, <sent_asignacion>

Atributos involucrados	**var_control.lexema**: *heredado* del IDENTIFICADOR que sigue a `pa_cada`. **var_control.escribible**: *heredado* hacia el ámbito del bucle. Dominio: {verdadero, falso}; vale **falso** para la variable de control y verdadero para cualquier otra. **var_control.categoria**: *sintetizado* al declararla. Dominio: {bucle}. **var_control.type**: *sintetizado* = numerito.
Restricción	Ninguna sentencia del cuerpo puede volver a enlazar la variable de control: ni `pille_pues i = …`, ni `escuche_pues(i)`, ni `i = …`. El atributo se consulta antes de cada redefinición: solo se admite si **escribible = verdadero**. Fuera del bucle el nombre vuelve a estar libre, porque el ámbito del `pa_cada` se cierra al terminar.
Acciones del compilador	1. Al entrar al `pa_cada`, abrir un ámbito nuevo e insertar la variable de control con categoría = bucle, tipo = numerito, inicializada = verdadero y **escribible = falso**. 2. En cada `pille_pues` y cada `escuche_pues` del cuerpo, buscar el nombre en la pila de ámbitos y leer su atributo escribible. 3. Si es falso, reportar error con fila, columna y lexema, y sugerir otro nombre para la variable nueva. 4. Cerrar el ámbito al salir del bucle y continuar. *Por qué importa para Gleam:* la variable de control se vuelve el parámetro de `fn(i) { … }` dentro de `list.each`; reenlazarla rompería el recorrido.

```
<paso_opcional>          ::= de_a <expresion>
                           | e
```

REGLA SEM-8 — Paso del pa_cada distinto de cero y recorrido alcanzable

Nodos de la gramática: <sent_para>, <paso_opcional>

Atributos involucrados	**valor(desde)**, **valor(hasta)**, **valor(paso)**: *sintetizados* por plegado de constantes sobre cada <expresion>. Dominio: $\mathbb{Z} \cup \{\bot\}$, donde $\bot$ significa «no es un literal» (admite el menos unario y los paréntesis). **paso_implicito**: *heredado*; vale 1 cuando <paso_opcional> deriva en ε. **direccion**: *sintetizado* en <sent_para> = signo(valor(paso)). Dominio: {+1, −1}.
Restricción	**valor(paso) ≠ 0**: con paso 0 la variable de control nunca avanza. Si **valor(paso) > 0** entonces **valor(desde) ≤ valor(hasta)**. Si **valor(paso) < 0** entonces **valor(desde) ≥ valor(hasta)**. Las dos últimas solo se exigen cuando los tres valores son literales: con límites variables el recorrido no es decidible en tiempo de compilación y la regla calla en lugar de dar un falso positivo.
Acciones del compilador	1. Plegar las tres expresiones a su valor constante; si <paso_opcional> es ε, usar paso_implicito = 1. 2. Si valor(paso) = 0, reportar error sobre el lexema del paso con fila y columna. 3. Si los tres son literales, comparar el signo del paso con el orden de los límites y reportar «el cuerpo nunca se ejecuta» indicando los tres valores concretos. 4. Si alguno no es literal, no reportar nada y seguir. 5. Continuar con el cuerpo del bucle. *Por qué importa para Gleam:* `list.range` no admite paso, así que con paso ≠ 1 el generador emite recursión de cola; un paso 0 produciría una recursión infinita.

```
/* ------------------------------------------------------------- */
/* 3.7  Repeticion con condicion al final  (do-while / repeat-until)*/
/* ------------------------------------------------------------- */

<sent_repita>            ::= repita_pues
                                 <bloque>
                             hasta_que <expresion>
```

REGLA SEM-6 — Terminación del bucle: la condición tiene que poder cambiar

Nodos de la gramática: <sent_mientras>, <sent_repita>

Atributos involucrados	**leidas(cond)**: *sintetizado* en la expresión de control. Dominio: conjunto de identificadores que la condición consulta (sin los nombres de función). **redefinidas(cuerpo)**: *sintetizado* en <bloque>. Dominio: conjunto de identificadores que el cuerpo vuelve a enlazar con `pille_pues` o `escuche_pues`. **valor(cond)**: *sintetizado* por plegado de constantes. Dominio: {sizas, naranjas} ∪ {⊥} (⊥ = no constante). **continua_si**: *heredado* de la construcción. Dominio: {verdadero} para `mientras_que`, {falso} para `repita_pues`, porque `hasta_que` es la condición de SALIDA, no de permanencia.
Restricción	**valor(cond) ≠ continua_si**: la condición no puede ser el literal que mantiene el bucle vivo (`mientras_que sizas` o `hasta_que naranjas`). **leidas(cond) ≠ ∅**: una condición sin variables tiene siempre el mismo valor. **leidas(cond) ∩ redefinidas(cuerpo) ≠ ∅**: el cuerpo debe redefinir al menos una de las variables que la condición lee. La regla es *conservadora y sólida en este lenguaje*: Paisascript es inmutable y sin efectos laterales, así que si esa intersección es vacía ninguna ejecución puede cambiar el valor de la condición.
Acciones del compilador	1. Plegar la condición: si es un literal booleano, compararlo con continua_si y reportar «el bucle nunca termina». 2. Recorrer la condición y recolectar leidas(cond), descartando los identificadores que la tabla de símbolos marca como función. 3. Recorrer el cuerpo (incluidos los bucles anidados) y recolectar redefinidas(cuerpo) a partir de los nodos <sent_declaracion>, <sent_lectura> y <sent_asignacion>. 4. Intersecar ambos conjuntos. Si el resultado es vacío, reportar error con fila, columna y lexema del bucle, nombrando las variables de la condición que habría que hacer avanzar. 5. Continuar el análisis del resto del programa. *Por qué importa para Gleam:* ese bucle se traduce a una recursión de cola; sin progreso, la función auxiliar nunca alcanza su caso base.

```
/* El cuerpo se ejecuta AL MENOS UNA VEZ y hasta_que da la condicion de  */
/* SALIDA (no de permanencia): el bucle repite mientras la expresion sea */
/* falsa. hasta_que cierra el bloque, de modo que no hace falta una      */
/* palabra de cierre adicional. La condicion se evalua DENTRO del ambito */
/* del cuerpo, asi que puede leer las variables que el cuerpo acaba de   */
/* crear: es el mismo estado que el generador pasa a la recursion de     */
/* cola de Gleam.                                                       */
/* -------------------------------------------------------------------- */
/* 3.8  Calce de patrones  (equivale al `case` de Gleam)                */
/* -------------------------------------------------------------------- */

<sent_pillemos>         ::= pillemos <expresion> "{" <lista_casos> "}"

<lista_casos>           ::= <caso> <lista_casos_resto>

<lista_casos_resto>     ::= <caso> <lista_casos_resto>
                          | e

<caso>                  ::= <patron> pa_que_lleve <cuerpo_caso>

<cuerpo_caso>           ::= "{" <bloque> "}"
                          | <sentencia>

<patron>                ::= NUM_ENTERO
                          | NUM_REAL
                          | CADENA_LITERAL
                          | sizas
                          | naranjas
                          | IDENTIFICADOR      /* enlaza el valor (catch-all) */
                          | "_"                /* comodin, descarta el valor  */
```

```
/* ------------------------------------------------------------------ */
/* 3.9  Expresiones                                                   */
/*      Jerarquia de precedencia, de menor a mayor:                   */
/*        o_que < y_tambien < igualdad < relacional < <>              */
/*              < +,- < *,/,% < ** < unarios < primaria               */
/*      Toda recursividad izquierda fue eliminada con la tecnica      */
/*      A ::= B A'   /   A' ::= op B A' | e                           */
/* ------------------------------------------------------------------ */

<expresion>             ::= <expr_o>

<expr_o>                ::= <expr_y> <expr_o_p>
<expr_o_p>              ::= o_que <expr_y> <expr_o_p>
                          | e

<expr_y>                ::= <expr_igualdad> <expr_y_p>
<expr_y_p>              ::= y_tambien <expr_igualdad> <expr_y_p>
                          | e

<expr_igualdad>         ::= <expr_relacional> <expr_igualdad_p>
<expr_igualdad_p>       ::= <op_igualdad> <expr_relacional> <expr_igualdad_p>
                          | e
<op_igualdad>           ::= igualito
                          | distinto

<expr_relacional>       ::= <expr_concat> <expr_relacional_p>
<expr_relacional_p>     ::= <op_relacional> <expr_concat> <expr_relacional_p>
                          | e
<op_relacional>         ::= ">"
                          | "<"
                          | ">="
                          | "<="

<expr_concat>           ::= <expr_aditiva> <expr_concat_p>
<expr_concat_p>         ::= "<>" <expr_aditiva> <expr_concat_p>
                          | e

<expr_aditiva>          ::= <expr_multiplicativa> <expr_aditiva_p>
<expr_aditiva_p>        ::= "+" <expr_multiplicativa> <expr_aditiva_p>
                          | "-" <expr_multiplicativa> <expr_aditiva_p>
                          | e

<expr_multiplicativa>   ::= <expr_potencia> <expr_multiplicativa_p>
<expr_multiplicativa_p> ::= "*" <expr_potencia> <expr_multiplicativa_p>
                          | "/" <expr_potencia> <expr_multiplicativa_p>
                          | "%" <expr_potencia> <expr_multiplicativa_p>
                          | e
```

REGLA SEM-3 — Compatibilidad de operandos en operadores

Nodos de la gramática: <expr_aditiva>, <expr_multiplicativa>, <expr_potencia>, <expr_unaria>, <expr_relacional>, <expr_igualdad>, <expr_y>, <expr_o>, <expr_concat>

Atributos involucrados	**type_izq** y **type_der**: *sintetizados* en los dos operandos. Dominio: tipos del lenguaje ∪ {desconocido, error}. **type**: *sintetizado* en el nodo del operador (tipo del resultado).
Restricción	**+ − * / ****: type_izq = type_der y ambos ∈ {numerito, quebradito}. **%**: type_izq = type_der = numerito. **> < >= <=**: operandos numéricos del mismo tipo; el resultado es siono. **y_tambien / o_que / nanai**: operandos siono; resultado siono. **<>**: operandos cuento, numerito o quebradito; resultado cuento (Paisascript convierte a texto al generar Gleam).
Acciones del compilador	1. Comparar type_izq con type_der según la tabla anterior. 2. Si no son compatibles, reportar el operador, los dos tipos y la posición (fila, columna, lexema). 3. Sintetizar el tipo del resultado o **error**; el tipo error es compatible con todo para no generar errores en cascada. 4. Continuar con el resto de la expresión.

```
/* La potencia asocia a la DERECHA: 2 ** 3 ** 2  =  2 ** (3 ** 2)   */
<expr_potencia>         ::= <expr_unaria> <expr_potencia_p>
<expr_potencia_p>       ::= "**" <expr_potencia>
                          | e

<expr_unaria>           ::= nanai <expr_unaria>
                          | "-" <expr_unaria>
                          | <expr_primaria>

<expr_primaria>         ::= NUM_ENTERO
                          | NUM_REAL
                          | CADENA_LITERAL
                          | sizas
                          | naranjas
                          | IDENTIFICADOR <sufijo_id>
                          | "(" <expresion> ")"

/* Factorizacion por la izquierda: variable y llamada comparten el   */
/* prefijo IDENTIFICADOR, por eso se separa el sufijo.               */
<sufijo_id>             ::= "(" <argumentos> ")"
                          | e

<argumentos>            ::= <arg_lista>
                          | e

<arg_lista>             ::= <expresion> <arg_resto>

<arg_resto>             ::= "," <expresion> <arg_resto>
                          | e
```

REGLA SEM-5 — Llamada a función: existencia, aridad y tipos

Nodos de la gramática: <sent_llamada>, <sufijo_id> dentro de <expr_primaria>, <argumentos>

Atributos involucrados	**arity**: *sintetizado* (número de argumentos). **arg_type[1..n]**: *sintetizados* en cada <expresion>. **param_type[1..n]** y **return_type**: consultados en la firma de la tabla de símbolos. **type**: *sintetizado* en la llamada = return_type.
Restricción	La función debe estar declarada; **arity = |params|**; **arg_type[i] = param_type[i]** para todo i; el tipo del resultado es el tipo de retorno declarado o inferido.
Acciones del compilador	1. Buscar la firma de la función en el ámbito global. 2. Si no existe, reportar «función no declarada» con su posición. 3. Comparar la aridad y reportar el número esperado y el recibido. 4. Comparar cada argumento con su parámetro formal y reportar los desajustes de tipo. 5. Sintetizar return_type y continuar.

==========

3. Implementación en el compilador (Parte 2)

El análisis semántico está implementado como una **fase claramente diferenciada** del análisis sintáctico: el módulo *analizador_semantico.py* recibe el CST que produce el parser y lo recorre con un **visitor**. El parser no contiene lógica semántica; la interfaz (*app.py*) invoca la fase después del análisis sintáctico.

```
from analizador_semantico import AnalizadorSemantico

analizador = AnalizadorSemantico(cst)      # CST del parser
errores = analizador.analizar()            # lista de ErrorSemantico
tabla = analizador.tabla                   # tabla de símbolos
```

3.1 Tabla de símbolos

Estructura de datos con **pila de ámbitos** (uno por bloque, función, `mientras_que`, `pa_cada`, `repita_pues` y `pillemos`). Cada entrada almacena:

- **lexema**, **tipo** (numerito, quebradito, cuento, siono o desconocido),
- **categoría** (variable, parámetro, función, variable de bucle o patrón),
- **ámbito** y **nivel** de anidamiento,
- **estado de inicialización**,
- **escribible** (falso solo para la variable de control de un `pa_cada`; es el atributo heredado que usa SEM-7),
- **fila** y **columna** de la declaración,
- para funciones: **parámetros** (nombre y tipo) y **tipo de retorno**.

La búsqueda de nombres va de dentro hacia afuera (alcance léxico), de modo que el *shadowing* es válido: una declaración nueva oculta a la anterior sin modificarla.

3.2 Detección y reporte de errores

Cada violación produce un objeto `ErrorSemantico` con **fila**, **columna**, **lexema**, la regla violada (SEM-1 … SEM-8) y un mensaje descriptivo. Ejemplo real de salida:

```
Error Semántico en [Fila 16, Columna 21] sobre 'edad':
  [SEM-1] Se declara 'edad' como numerito, pero la expresión es de tipo cuento

Error Semántico en [Fila 20, Columna 27] sobre '+':
  [SEM-3] No se pueden mezclar 'quebradito' y 'numerito' con '+':
  Gleam no convierte implícitamente entre numerito y quebradito

Error Semántico en [Fila 22, Columna 12] sobre 'saldo_disponible':
  [SEM-2] El identificador 'saldo_disponible' no ha sido declarado en el
  ámbito actual ni en uno ancestro

Error Semántico en [Fila 24, Columna 33] sobre 'sumar':
  [SEM-5] La función 'sumar' espera 2 argumento(s) y se pasaron 1
```

3.3 Continuación del análisis (recuperación)

El visitor **no se detiene** en el primer error. Tras reportar una violación continúa recorriendo el resto del árbol; las expresiones inválidas sintetizan el tipo especial `error`, que es compatible con cualquier otro tipo para evitar errores en cascada. Por eso un programa con varios fallos reporta **todos** en una sola pasada (verificado en *test_semantico.py*).

3.4 Separación de fases

Fase	Módulo	Responsabilidad
Léxica	lexer.py	Tokens con lexema, fila y columna.
Sintáctica	parser.py	CST según la gramática BNF (LL(1) recursivo).
Semántica	**analizador_semantico.py**	Visitor sobre el CST, tabla de símbolos y reglas SEM-1 … SEM-8.

4. Bonus — Integración de IA (Modalidad A)

Modalidad elegida: A — IA como capa adicional de validación. Los errores que detectan las reglas semánticas clásicas se envían a un modelo de lenguaje (OpenAI, configurable por `OPENAI_API_KEY` en `.env` o en los Secrets de Streamlit) junto con el código fuente, la tabla de símbolos y la gramática BNF. El modelo devuelve una explicación de la causa y una propuesta de corrección en lenguaje natural.

Flujo de integración:

1. El compilador ejecuta las reglas clásicas (SEM-1 … SEM-8).
2. Si hay errores, la interfaz ofrece el botón «Explicar y corregir estos errores semánticos con IA».
3. `ai_assistant.analizar_error_semantico_con_ia(codigo, errores, tabla)` construye el prompt con el código, los errores y la tabla de símbolos.
4. La respuesta se muestra como sugerencia. **Las reglas clásicas siguen siendo la fuente de verdad**: la IA no las reemplaza, las complementa.

```
from ai_assistant import analizar_error_semantico_con_ia

sugerencia = analizar_error_semantico_con_ia(
    codigo, errores_semanticos, tabla_simbolos, api_key=api_key
)
```

4.1 Ejemplo concreto

Código fuente (fragmento):

```
pille_pues numerito edad = "veinte anios"
```

Error original detectado por la regla clásica:

```
[SEM-1] Se declara 'edad' como numerito,
pero la expresión es de tipo cuento
(fila 16, columna 21, lexema 'edad')
```

Sugerencia generada por la IA (ejemplo de salida):

🔍 **Causa:** la variable `edad` se anotó como `numerito` (entero), pero la expresión inicializadora es un literal de tipo `cuento` (cadena). Paisascript tiene tipado estático y fuerte: no convierte cadenas a números de forma implícita.

🛠️ **Corrección propuesta:**

```
pille_pues numerito edad = 20
```

Si el valor debe leerse de consola, use `escuche_pues(edad)`, que declara e inicializa el identificador.

📌 **Regla aplicada:** SEM-1 — declaración y compatibilidad de tipos.

los hipopótamos bailan desnudos en la playa